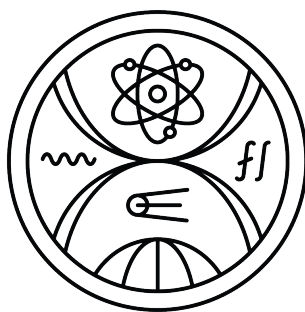


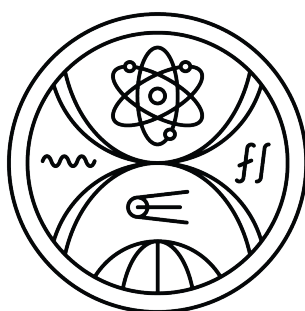
COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS, PHYSICS AND INFORMATICS



ENVIRONMENT FOR CREATING MUSIC USING PROGRAMMING

Master's thesis

COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS, PHYSICS AND INFORMATICS



ENVIRONMENT FOR CREATING MUSIC USING PROGRAMMING

Master's thesis

Study program: Applied informatics
Branch of study: Applied informatics
Department: Department of Applied Informatics
Supervisor: doc. RNDr. Ľubomír Salanci, PhD.



ZADANIE ZÁVEREČNEJ PRÁCE

- Meno a priezvisko študenta:** Bc. Maksim Kulagin
Študijný program: aplikovaná informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: diplomová
Jazyk záverečnej práce: anglický
Sekundárny jazyk: slovenský
- Názov:** Environment for creating music using programming
Prostredie na tvorbu hudby pomocou programovania
- Anotácia:** Práca patrí do oblasti hudobného programovania a algoritmickej tvorby hudby. Táto oblasť umožňuje opisovať hudobné štruktúry pomocou formálnych jazykov a vytvárať hudbu prostredníctvom textového kódu.
- Cieľ:** Cieľom práce je návrh a implementácia programovacieho jazyka a vhodného prostredia pre tvorbu zvukov a hudby. Textovo orientovaný programovací jazyk spolu s výsledným prostredím umožňujú popisovať hudobné štruktúry (vzory, tóny, rytmus a ďalšie parametre) vo forme zdrojového kódu, ktorý bude následne spracovaný do internej reprezentácie tak, aby bolo možné výsledok efektívne prehrávať. Súčasťou práce je prehľad prístupov k tvorbe hudby prostredníctvom aktuálnych moderných technológií, analýza vybraných existujúcich hudobných programov, programovacích jazykov a knižníc. Výskum je zameraný na použiteľnosť jazyka a prostredia v cieľovej skupine na sade modelových hudobných prípadov.
- Literatúra:** Mernik, M., Heering, J., Sloane, A. M. When and How to Develop Domain-Specific Languages. ACM Computing Surveys, Vol. 37, No. 4, pp. 316-344, 2005. <https://doi.org/10.1145/1118890.1118892>
Aaron, S., Blackwell, A. F. From Sonic Pi to Overtone: Creative Musical Experiences with Domain-Specific and Functional Languages. Proceedings of the 1st ACM SIGPLAN Workshop on Functional Art, Music, Modelling and Design (FARM), pp. 35-46, 2013. <https://doi.org/10.1145/2505341.2505346>
McCartney, J. Rethinking the Computer Music Language: SuperCollider. Computer Music Journal, Vol. 26, No. 4, pp. 61-68, 2002. <https://doi.org/10.1162/014892602320991383>
McLean, A. Making Programming Languages to Dance to: Live Coding with Tidal. Proceedings of the 2nd ACM SIGPLAN Workshop on Functional Art, Music, Modelling and Design (FARM), pp. 63-70, 2014. <https://doi.org/10.1145/2633638.2633647>



Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Vedúci: doc. RNDr. Ľubomír Salanci, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.
Dátum zadania: 15.12.2015

Dátum schválenia: prof. RNDr. Roman Ďurikovič, PhD.
garant študijného programu

.....
šľudent

.....
vedúci práce

Abstract

Music programming is the practice of creating music by writing executable code. Our goal was to make a standalone environment for creating music using programming. The system is built around a custom text-based domain-specific language that allows users to describe musical patterns in the form of source code. The language is processed by a hand-written compiler front-end consisting of a lexer, a recursive-descent parser and a typed abstract syntax tree. A tree-walking interpreter transforms the AST into a list of timed musical events, which are played back through an audio backend. Several existing music programming environments were analysed during the design phase. The graphical interface is implemented in WPF.

Keywords: music programming, domain-specific language, algorithmic composition

Abstrakt

Hudobné programovanie je prax tvorby hudby písaním spustiteľného kódu. Naším cieľom bolo vytvoriť samostatné prostredie na tvorbu hudby pomocou programovania. Systém je postavený na vlastnom textovom doménovo-špecifickom jazyku, ktorý umožňuje používateľom opisovať hudobné patterny vo forme zdrojového kódu. Jazyk je spracovaný ručne napísaným prekladačom pozostávajúcim z lexera, rekurzívneho zostupného parsera a typovaného abstraktného syntaktického stromu. Stromový interpreter transformuje AST na zoznam časovaných hudobných udalostí, ktoré sú prehrávané prostredníctvom audio backendu. Počas návrhu bolo analyzovaných niekoľko existujúcich prostredí na hudobné programovanie. Grafické rozhranie je implementované v technológii WPF.

Kľúčové slová: hudobné programovanie, doménovo-špecifický jazyk, algoritmická tvorba hudby

Contents

1	Introduction	1
1.1	Overview	1
1.2	Music programming	1
1.3	Algorithmic composition	2
2	Music programming languages	3
2.1	Domain-specific languages	3
2.1.1	Definition	3
2.1.2	Implementation approaches	4
2.2	Existing systems	4
2.2.1	SuperCollider	4
2.2.2	Sonic Pi	5
2.2.3	TidalCycles	6
2.3	Comparison	6
3	Proposed solution	8
3.1	Requirements	8
3.2	Architecture overview	10

Terminology and Abbreviations

Terms

- **Algorithmic composition**

The technique of using algorithms to create music.

- **Domain-specific language**

A computer language specialised to a particular application domain, in contrast to a general-purpose language, which is broadly applicable across domains.

- **External DSL**

A DSL with its own custom syntax and a dedicated parser, written and processed independently of any host language.

- **Embedded DSL**

A DSL implemented as a library within a general-purpose host language, so that DSL programs are valid programs of that host language; also known as an internal DSL.

- **Live coding**

A performing arts form and a creativity technique centred upon the writing of source code and the use of interactive programming in an improvised way, often applied to sound, image and other digital media.

- **Sampling**

In sound and music, the reuse of a portion (or sample) of a sound recording in another recording.

- **Envelope**

In sound and music, a description of how a sound changes over time; envelopes are commonly characterised by attack, decay, sustain and release stages.

- **Recursive descent parser**

A kind of top-down parser built from a set of mutually recursive procedures, in which each procedure implements one of the nonterminals of the grammar.

Abbreviations

- **AST** – Abstract Syntax Tree.
- **BPM** – Beats Per Minute.
- **DAW** – Digital Audio Workstation.
- **DSL** – Domain-Specific Language.
- **MIDI** – Musical Instrument Digital Interface.
- **OSC** – Open Sound Control.
- **PCM** – Pulse-Code Modulation.

Motivation

Music has always been a creative discipline that requires years of training to master. At the same time, programming has become an increasingly accessible skill taught at every level of education. The intersection of these two fields — music programming — offers a unique opportunity: allowing people to create music through code, without the need for traditional musical training or expensive instruments.

The growing interest in algorithmic composition has produced a number of music programming environments. These systems have demonstrated that a small, well-designed language can serve as a powerful creative tool. However, most existing solutions require significant technical prerequisites — external audio servers, knowledge of specific programming paradigms, or complex installation procedures — which creates a barrier for beginners and educators alike.

Our thesis addresses that gap by presenting a self-contained music programming environment designed for users who are new to either programming or music, running natively without any external dependencies.

Chapter 1

Introduction

1.1 Overview

This chapter introduces the two fields in which the present work is rooted: music programming, the practice of producing and manipulating sound through code, and algorithmic composition, the partial or total automation of the act of composing music. The aim is to give a short, self-contained overview of each field and to show how their open problems motivate the rest of the thesis.

1.2 Music programming

Music programming is the practice of producing or manipulating musical material through executable code. A short script can generate a MIDI file, a larger program can drive a hardware synthesiser through a control interface, and a live coder can edit source code on stage and let the audience hear the result. What these activities share is that the primary representation of the music is a text—a program—rather than a score, a piano roll or a set of audio clips.

Music-programming systems fall into three overlapping categories. Offline systems, such as the MUSIC-N family, describe a score and an orchestra, render an audio file and play it back afterwards. Real-time systems, of which SuperCollider [?] is the most influential example, produce sound while the program executes, enabling algorithmic improvisation and interactive composition. Live-coding systems take this one step further: the performer modifies the source of a running program in front of an audience, and every edit audibly changes the music; well-known examples are Sonic Pi [?] and TidalCycles [?].

These categories share the same engineering problems. The abstraction level must be high enough to be musically meaningful but low enough to control timing precisely. The runtime must align the logical time of the score with the wall-clock time of the

audio hardware. The language must express repetition and variation concisely, because music consists largely of patterns and their transformations. The environment must give fast auditory feedback, so the composer can hear the consequences of a change within a fraction of a second.

These problems are exactly the kind that domain-specific languages address [?]. A general-purpose language can solve them, but it forces the user to reinvent the musical abstractions on top of generic constructs every time. A DSL encodes the musical domain directly. That observation is the starting point of this thesis.

1.3 Algorithmic composition

Algorithmic composition is the partial or total automation of the process of composing music [?]. The word “partial” is important: in most practical systems the computer does not replace the composer but operates on a well-defined subtask, such as generating a melody over a given harmony, expanding a rhythmic seed, or harmonising a melody by a set of rules. The composer keeps the overall aesthetic decisions and treats the algorithm as a tool.

The same survey groups the techniques into formal grammars, knowledge-based systems, Markov and other stochastic models, evolutionary algorithms and neural networks, which working systems usually combine. A complementary line of work, exemplified by TidalCycles, looks for an algebraic foundation of music itself, in which patterns are first-class values that can be combined and transformed with a small set of operators.

Our thesis sits in the more modest space of computer-aided algorithmic composition: the proposed language offers a small set of primitives that make pattern- and rule-based ideas easy to express. The detailed motivation for these design choices is developed in Chapter 3.

Chapter 2

Music programming languages

The previous chapter argued that music programming and algorithmic composition benefit from languages whose abstractions are tailored to the musical domain. This chapter develops that argument in detail. It first revisits the notion of domain-specific language from a language engineering perspective and surveys the main implementation strategies that have shaped the field. It then examines three well-known music programming systems—SuperCollider, Sonic Pi and TidalCycles—each of which embodies a distinct combination of host language, abstraction style and runtime architecture. The chapter concludes with a comparative summary that motivates the design space later occupied by the proposed solution.

2.1 Domain-specific languages

2.1.1 Definition

A domain-specific language (DSL) is a programming language “tailored to a specific application domain” [?]. Its defining property is not size or implementation technique but that its abstractions, syntax and semantics are chosen with reference to a single domain. DSLs are contrasted with general-purpose languages (GPLs) such as Java or Python, whose abstractions are deliberately neutral. In their target domain, DSLs “offer substantial gains in expressiveness and ease of use” [?]: a few lines of domain code replace dozens of equivalent GPL lines, programs read closer to the way domain experts think, and many errors are ruled out by construction.

Common examples include BNF for grammars, SQL for queries, HTML and LaTeX for documents, Make for builds, VHDL for hardware. Music programming languages—SuperCollider [?], Sonic Pi [?] and TidalCycles [?]—fit naturally into this list: their domain is sound, time and musical structure.

2.1.2 Implementation approaches

Mernik et al. enumerate several common implementation patterns. The three that matter for music programming are: **interpreter** (the DSL is parsed to an AST and evaluated directly; Sonic Pi uses this strategy in Ruby), **compiler** (the DSL is translated to a lower-level representation; SuperCollider’s slang JIT-compiles to bytecode), and **embedding** (the DSL is a library inside a host language; TidalCycles is embedded in Haskell, Sonic Pi in Ruby).

The choice of strategy is rarely orthogonal to the choice of abstractions. A functional, pattern-based language like Tidal benefits from a lazy host. A language that describes imperative sequences of musical events, such as the one proposed in this thesis, is naturally implemented as a tree walker over an AST. The proposed solution follows the latter pattern: an external DSL with its own grammar, scanner, parser, AST and interpreter that emits events for a dedicated audio renderer (Chapter 3).

2.2 Existing systems

Having outlined the DSL design space in the previous section, we now turn to three concrete music programming languages that occupy distinct points within it. SuperCollider is the senior member of the family, a standalone language and audio server combined; Sonic Pi is an embedded DSL designed for pedagogy; TidalCycles is an embedded functional DSL focused on pattern-based live coding.

2.2.1 SuperCollider

SuperCollider, described in detail by McCartney in 2002 [?], is the senior member of the family and one of the most influential music programming environments to date. It is split into two cooperating processes: the language interpreter slang and the real-time audio server scsynth, which communicate over Open Sound Control. This decoupling lets synthesis run in a real-time priority process while the language, free of audio deadlines, handles composition and user interaction.

The language is dynamically typed and object-oriented, with a Smalltalk-derived syntax. On top of that core, SuperCollider provides a rich library of unit generators (UGens), the building blocks of digital signal processing. The user constructs synthesis graphs by composing UGens, and the server processes them at block or single-sample granularity. A short example:

```
SynthDef(\sine, { |freq=440, amp=0.2|
  Out.ar(0, SinOsc.ar(freq) * amp)
}).add;
```



```
Pbind(
  \instrument, \sine,
  \degree, Pseq([0, 2, 4, 7], inf),
  \dur, 0.25
).play;
```

Listing 2.1: A minimal SuperCollider example.

Limitations. The expressive power comes at a steep cost. The user must learn a Smalltalk-derived object-oriented language, the UGen graph algebra, and the asynchronous OSC protocol that connects them. Installation requires keeping scsynth alive as a separate process; runtime errors cross the language/server boundary and are hard to diagnose. There is no beginner-oriented IDE in the default distribution.

2.2.2 Sonic Pi

Sonic Pi, developed by Sam Aaron at the University of Cambridge, is a DSL and IDE designed for teaching programming and music to young learners on the Raspberry Pi [?]. It targets users with no prior experience in programming or music theory: the surface syntax is small, the IDE provides a fixed set of panes, and audible feedback is available within seconds of the first line of code.

Architecturally, Sonic Pi is an internal DSL embedded in Ruby. The user’s program is parsed and evaluated as Ruby, but the operative vocabulary consists of a few domain-specific functions—most importantly `play` (trigger a note), `sleep` (advance musical time) and `in_thread` (run parallel musical processes). Audio synthesis is delegated to a background SuperCollider scsynth instance via OSC, but from the user’s point of view SuperCollider is invisible. A minimal program:

```
use_bpm 120
4.times do
  play :c4, release: 0.4
  sleep 0.5
  play :e4, release: 0.4
  sleep 0.5
  play :g4, release: 0.4
  sleep 0.5
end
```

Listing 2.2: A minimal Sonic Pi program.

Limitations. Sonic Pi hides SuperCollider’s complexity but inherits its dependencies: a separate scsynth process must be installed and kept alive, and the Ruby host

adds a second runtime. The fixed five-pane IDE optimises for novices but cannot be customised. The toolchain is most polished on Raspberry Pi; on Windows it remains secondary.

2.2.3 TidalCycles

TidalCycles, designed by Alex McLean [?], is a DSL for the live coding of musical patterns. Tidal is an internal DSL embedded in Haskell, and it differs from SuperCollider and Sonic Pi in a fundamental way: its central abstraction is not the individual note or the imperative sequence of events, but the pattern.

In Tidal, time is modelled as an infinite stream of cycles. A pattern is a function that, given a time interval, returns the events that fall inside it. Patterns are therefore first-class values in Haskell, and the language equips them with a rich algebra of combinators (`fast`, `slow`, `rev`, `every`, `cat`, `stack`, ...) that build new patterns by manipulating time and structure rather than individual events. Tidal does not produce sound by itself: patterns are sent over OSC to an audio back-end (typically SuperDirt on top of `scsynth`). A short example:

```
d1 $ every 3 (fast 2)
    $ stack [ sound "bd sn bd sn"
              , sound "hh*8"
              , n "0 3 5 7" # sound "piano"
            ]
```

Listing 2.3: A short TidalCycles pattern.

Limitations. Tidal requires installing a working Haskell toolchain, the Tidal library and a SuperCollider back-end; on Windows this has historically been the most fragile of the three systems. There is no integrated IDE—the workflow takes place in a text editor wired to a REPL through a plug-in. From the language side, the pattern-as-function model is mathematically elegant but does not map onto how most musicians verbalise music, which makes Tidal an unusual first exposure for a learner.

2.3 Comparison

The three systems surveyed in the previous section share a common goal, the production of sound through code, but they realise that goal through very different combinations of host language, paradigm, runtime architecture and target audience. Table 2.1 summarises the dimensions along which they differ and adds the proposed solution of this thesis in the same frame of reference. The aim is not to rank the systems but to make

their design choices and their main drawbacks explicit, so that the niche the proposed project inhabits becomes visible.

Table 2.1: Comparison of music programming languages along the dimensions of host, paradigm, audio back-end, target audience and main drawback.

System	Host / base	Paradigm	Audio back-end	Target audience	Main drawback
SuperCollider	Standalone (sclang)	Object-oriented, Smalltalk-like	scsynth server (built in)	Experienced composers / DSP programmers	Steep learning curve; dual-process runtime; no novice IDE
Sonic Pi	Ruby embedding	Imperative, threaded	SuperCollider (scsynth)	Beginners, classroom	Inherits external SuperCollider server; Ruby runtime; fixed IDE
TidalCycles	Haskell embedding	Functional, pattern-based	SuperDirt (via OSC)	Live coders, electronic musicians	Requires Haskell + SuperDirt; no integrated IDE; pattern model unintuitive for beginners

Several observations follow from the table. First, all three reference systems delegate audio synthesis to a SuperCollider-derived server, even when the surface language has nothing in common with `sclang`. This delegation buys industrial-strength DSP at the cost of an extra process, an OSC bus and an external dependency that the user must install and keep alive. The proposed solution breaks with this convention and bundles a simple sine-wave renderer directly into the application.

Second, the three reference systems are spread across the embedding / standalone axis. SuperCollider is a standalone language with a bespoke runtime; Sonic Pi and TidalCycles are embedded DSLs that inherit their syntax from Ruby and Haskell respectively. Each approach has well-known advantages: embedding gives the user the host’s full type system, module system and tooling, while a standalone language can shape its grammar entirely around the target domain.

Third, the paradigms differ markedly. Tidal is functional and works on patterns as values; Sonic Pi and SuperCollider, despite their differing implementations, both present an imperative, time-ordered view of music to the user.

Chapter 3

Proposed solution

The two preceding chapters surveyed the field. This chapter turns to the proposed solution itself: a text-oriented music programming language and the accompanying desktop environment in which it is edited, compiled and played back. The chapter is organised in four parts. The first part lists the requirements that shaped the design, distinguishing the functional from the non-functional ones. The second part gives a high-level view of the architecture and the data flow from source code to audio. The third part discusses the language design principles that follow from those requirements and contrasts them, where relevant, with the systems reviewed in Chapter 2. The fourth part documents the implementation technology—the choice of C#, the WPF user interface and the NAudio audio back-end—and explains the trade-offs that led to it.

3.1 Requirements

The requirements that shaped the proposed solution fall into two categories. Functional requirements describe what the language and its environment are able to express; non-functional requirements describe the qualities the resulting system must exhibit, such as responsiveness or ease of installation.

Functional requirements

F1 – Notes and durations. The language must allow individual musical notes, identified by pitch and octave, to be triggered with a given duration. The set of notes must cover at least the full MIDI range of 0–127, expressed in scientific pitch notation (e.g. :c4, :f#3).

F2 – Tempo control. The user must be able to set the global tempo (in beats per minute) at any point in the program, so that all subsequent durations are inter-

preted relative to that tempo.

- F3 – Explicit temporal control.** In the spirit of Sonic Pi, the language must provide an explicit sleep primitive that advances logical musical time. This makes the order of musical events a direct consequence of the order of the source statements.
- F4 – Repetition.** A bounded repeat n do ... end construct must be available, so that fragments of music can be iterated a fixed number of times without textual duplication. A safety-capped loop construct (currently bounded at 32 iterations) is provided for prototyping.
- F5 – User-defined subroutines.** The language must support define *name* do ... end, allowing the user to give a name to a block of code and invoke it by writing the name as a statement. This is the minimum form of abstraction needed to compose larger pieces from smaller motifs.
- F6 – Playback.** The environment must be able to render the program’s musical events to the system’s default audio output, with a latency low enough to support an interactive edit/play workflow. The user must also be able to stop playback at any time.
- F7 – Examples.** The environment must ship with a small set of illustrative example programs that cover the basic features of the language, so that a new user can run a meaningful program within a minute of opening the application.

Non-functional requirements

- N1 – Low barrier to entry.** The combined surface area—grammar, primitives, IDE controls—must be small enough that a user with no prior programming experience can produce a simple melody within a short tutorial session.
- N2 – Readability.** Source code must read close to natural English: `play :c4, 0.4` should be intelligible without reference to a manual. Symbolic operators and implicit conversions are avoided where a keyword would do.
- N3 – Self-contained toolchain.** In contrast to Sonic Pi and TidalCycles, the proposed solution must not depend on a separate audio server, OSC bus or other background process. The whole pipeline—editor, scanner, parser, interpreter and audio renderer—ships as a single Windows desktop application.

The next section shows how these requirements are realised by the architecture of the implementation.

3.2 Architecture overview

The proposed solution is implemented as a single WPF desktop application written in C#. Internally the application is organised as a classical compiler-style pipeline, in which source code passes through a sequence of well-defined stages until it reaches the sound card. The five stages of the pipeline and the data that flows between them are summarised in Figure 3.1.

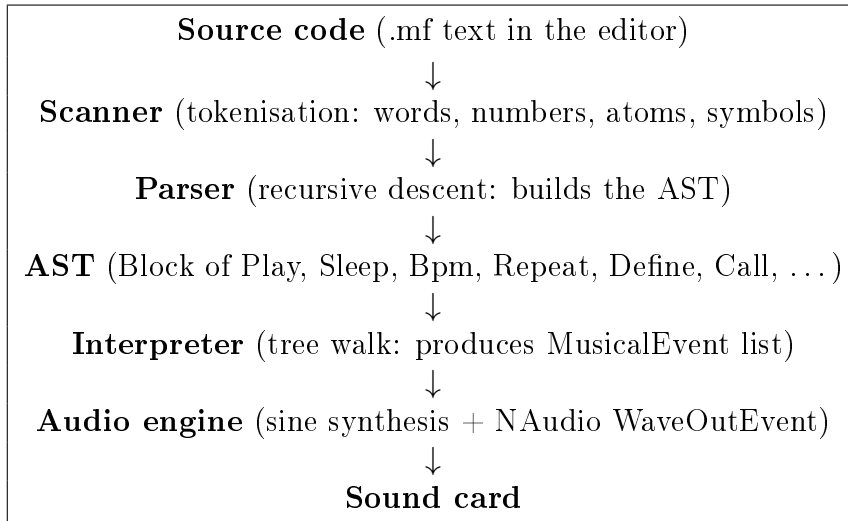


Figure 3.1: The processing pipeline of the proposed system, from source code in the editor to audio at the sound card.

Editor. The IDE is built with WPF and provides a minimal but sufficient set of controls: a text area for the source, buttons for loading the bundled examples, a Play button that triggers the whole pipeline, and a Stop button that halts playback. The dark theme is deliberate and aims to reduce visual fatigue during longer sessions of composition.

Scanner. The scanner is a hand-written character-by-character lexer. It skips whitespace and line comments introduced by `#`, then classifies the next token as a number, a word, an atom (anything beginning with a colon, such as `:c4` or `:piano`) or a single-character symbol. The scanner exposes its result through a small set of globals (token, kind, position) together with a `Check` helper that the parser uses to assert expectations and to emit helpful error messages with byte positions. Atoms are first-class in this lexer in order to give note names and instrument names a uniform, self-evident surface syntax.

Parser. The parser is a small recursive-descent translator from the token stream to the AST. Its entry point, `Parse`, returns a `Block` node containing one `Node` per top-

level statement; ParseStatement dispatches on the leading keyword (play, sleep, bpm, instrument, repeat, loop, define) and delegates to a dedicated sub-parser for the rest of the statement.

AST. The abstract syntax tree is a small hierarchy of Node subclasses, one per surface construct: Block, Const, Atom, Play, Sleep, Bpm, Repeat, Loop, Define, Call. Each node implements an Interpret method, so evaluation is performed by an ordinary recursive tree walk and no separate visitor infrastructure is required.

Interpreter. Evaluation maintains a handful of global fields—the current time (in beats), the current bpm, the current instrument, a list of MusicalEvents and a dictionary of user-defined subroutines—and exposes a Reset method that returns the interpreter to a clean state before each run. The semantics of the language are therefore very direct: play appends an event at the current beat; sleep x adds x beats to time; bpm n updates the global tempo; repeat n executes its body n times; define stores the body under a name; Call re-executes a stored body.

Audio back-end. The renderer is deliberately minimal. It scans the list of musical events produced by the interpreter, allocates a float buffer long enough to contain them all, and for each pitched event mixes a sine wave of the appropriate frequency—computed from the MIDI note via $f = 440 \cdot 2^{(m-69)/12}$ —with a short five-millisecond fade-in and fade-out envelope to suppress clicks. The buffer is normalised, converted to 16-bit PCM at 44,100 Hz and handed to NAudio’s WaveOutEvent for playback. The deliberate simplicity of this back-end is itself a design statement: by bundling synthesis directly into the application, the system satisfies the self-containedness requirement N3 (Section 3.1) at the cost of audio fidelity, which can be improved in later iterations without touching the language.

The pipeline as a whole gives a clean separation of concerns: the front-end (scanner, parser) is responsible for grammar; the AST is the single language artefact shared between the parser and the interpreter; the interpreter is responsible for semantics; the audio engine is responsible for sound. Each component can evolve independently, which also makes the system a useful didactic example of how a domain-specific language is built end-to-end.

The audio back-end is where the project diverges most visibly from SuperCollider, Sonic Pi and TidalCycles. Those systems delegate synthesis to a separate scsynth server reached over OSC. This system renders audio inside the same process as the interpreter, using NAudio [?].

NAudio exposes the Windows multimedia stack through .NET classes. Only two are needed: WaveFormat (sample rate, bit depth, channels) and WaveOutEvent (schedules

a stream of PCM samples for the default audio device). The renderer is simple:

1. Walk the event list and compute the total length in seconds.
2. Allocate a float buffer at 44.1 kHz of that length, plus a quarter-second tail of silence.
3. For each pitched event, derive $f = 440 \cdot 2^{(m-69)/12}$ from the MIDI number m , write a sine wave of that frequency at amplitude 0.6 into the buffer at the event's onset, and apply a 5 ms linear fade-in/out.
4. Normalise, convert to 16-bit signed PCM and hand the byte stream to WaveOutEvent.

This back-end is deliberately minimal: one timbre (a pure sine), no envelopes beyond the click-suppression fades, no polyphonic limiter beyond final normalisation. The benefit is self-containment (N3): no external server, no OSC bus, no port to keep open. The renderer is also replaceable without touching the language—a future iteration could swap the sine generator for a sample-playback engine, a subtractive synthesiser or VST plug-ins without altering scanner, parser, AST or interpreter.